Capstone Details (Phase 3)

Now that SecureChat has a strong foundational build for secure local communication, the **final phase** will focus on extending its functionality to operate **over the Internet** with **24/7 availability**!

Phase 3: SecureChat – Cloud-Hosted, Internet-Accessible Version

Objectives

- Transition SecureChat from a LAN-based app to an Internet-accessible, hosted solution.
- Ensure global accessibility and reliability via 24/7 hosting. There are free hosting sites you can utilize. One I can recommend is: **https://www.infinityfree.com/ (https://www.infinityfree.com/)** . You are free to use any cloud provider/self-hosting service you find best suited for your needs.
- Maintain strong encryption and security standards while adapting to web/cloud constraints.

Key Development Tasks

---

1. Backend Hosting & Deployment

- **Backend Transition**:
  - You may need to convert or adapt existing backend (likely Node.js or Python) to PHP-compatible endpoints OR decouple chat service using WebSocket server hosted elsewhere (see below).
  - Use InfinityFree to host frontend and API endpoints.
- **Database Setup**:
  - Set up **MySQL** backend on InfinityFree for storing users, messages (encrypted), logs (sanitized), etc.

- Implement **database connection pooling** and query sanitization to prevent SQL injection.

---

2. **WebSocket Server Deployment**

- If you decide to use InfinityFree, **InfinityFree doesn't support WebSockets or long-running backend processes** so you will need to deploy a WebSocket server using:
  - **Glitch**, **Render**, **Fly.io**, or **Railway** (all have free WebSocket support).
  - Use **InfinityFree** only for frontend and REST APIs (non-realtime data).
- Ensure secure communication over **WSS (WebSocket Secure)**.
- Add **heartbeat/ping mechanism** to maintain persistent connections.

---

3. **Public Web App Interface**

- **Rebuild frontend as a web application** (HTML/CSS/JavaScript):
  - Host on InfinityFree.
  - Secure using **HTTPS (via Cloudflare if needed, just a suggestion)**.
  - Integrate WebSocket client code for real-time messaging.
- Include:
  - Login & registration (with hashed password storage via backend).
  - Chat interface with emoji and file upload support.

---

4. **Secure File Sharing (Cloud-Friendly)**

- Host uploaded files on free cloud storage (e.g., Firebase Storage, Backblaze B2, or IPFS).
- Scan file uploads client-side before transmission for malware (basic validation).

---

5. **Encryption Enhancements**

- Use **End-to-End Encryption** on all messages and files before they leave the client.
- Store only **encrypted blobs** on the server. The server must not be able to decrypt messages.

---

6. **Online/Offline Presence Management**

- Implement presence detection: “online,” “offline,” “typing…” indicators via WebSocket ping/pong or database flags.

---

7. **24/7 Uptime Strategy**

- Use **uptime monitoring services** (like UptimeRobot) to ping backend and ensure it's online.
- Add **auto-reconnect** logic in the client for dropped WebSocket connections.
- **NOTE: Your service should remain up till May 25th for grading!**

---

8. **Scalability & Rate Limiting**

- Basic rate limiting on requests to prevent abuse.
- Consider CAPTCHAs or throttling for login/register APIs.

Hosting Tech Stack Overview

Component	Service	Notes
Frontend (HTML/CSS/JS)	InfinityFree	Static files and REST APIs
WebSocket Backend	Glitch / Render / Railway	Handles live chat
MySQL Database	InfinityFree MySQL	Encrypted message storage
File Storage	Firebase / Backblaze B2	For secure file transfers
Domain & SSL	Cloudflare + InfinityFree	Free SSL & DNS proxying

---

Testing & Rollout Plan

1. **Internal Testing** – Use testers in different locations.
2. **Security Testing** – Run penetration tests (Burp Suite) to validate encryption and input sanitization.
3. **Beta Release** – Deploy and collect user feedback. Ask fellow classmates to test your product out.
4. **Public Launch (Submission)** – Submit and watch your chat go live!